ECS 189C
Lecture 4:

Intro to interactive verification

Plan

Introduction to interactive program verification and why it matters

(Slides today; back to live coding after)

We know about

- Writing specifications in Python (testing / test harness method)
- Writing and solving/proving specifications in Z3

(Really needs new logo)

Main limitations of testing

Testing can only prove the property on **some** inputs, not on all inputs

Main limitations of verification with Z3

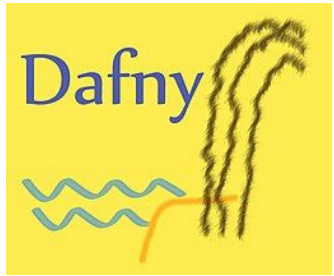
1. Z3 proves the property on **all** inputs, but requires rewriting the program in Z3.

2. Z3 can return “Unknown”

Examples we have seen

- Pigeonhole principle
- Arrays/functions - nontrivial properties (“function is one-to-one”)
- Chess moves puzzle
- Array property (time to file an issue report)

Z3 vs Dafny

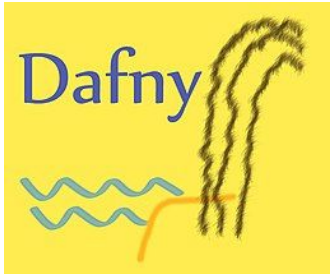


Interactive verification

Basically a more powerful/general way to write and verify programs.

- We can write more general specifications
- We can write the proofs ourselves - don't need to rely on the tools terminating or finding the proof automatically
  - Will allow us to solve all of the examples above
- We can incorporate verified code into bigger projects

^^^ more work + more effort = more payoff



Why use interactive verification?

So, you've written your code. You've tested it, and it seems to be working the way you expect.

> It's a lot of work to write specifications!
>
> It's a lot of work to prove specifications!

So when might you want to go the extra mile and do all this extra work?

Answer

Interactive verification is especially useful in cases where:

1. **Correctness is critical – bug causes catastrophic or irreversible consequences**
2. **Security – bug could be exploited by an attacker**
3. **Cost – a lot of $$$ is involved**

1. Correctness is critical

If the software fails, some very serious consequence will occur

Pentium bug

Intel, 1994: Bug in floating point

$$\frac{4{,}195{,}835}{3{,}145{,}727} = 1.333739068902037589$$

Pentium bug

Intel, 1994: Bug in floating point

- December 1994: Intel **recalls** all Pentium processors
- $475 million in losses

Incident led to renewed interest in formal verification:

today, chip design at companies like Intel and IBM is

validated by formal methods prior to deployment

1. Correctness is critical

If the software fails, some very serious consequence will occur

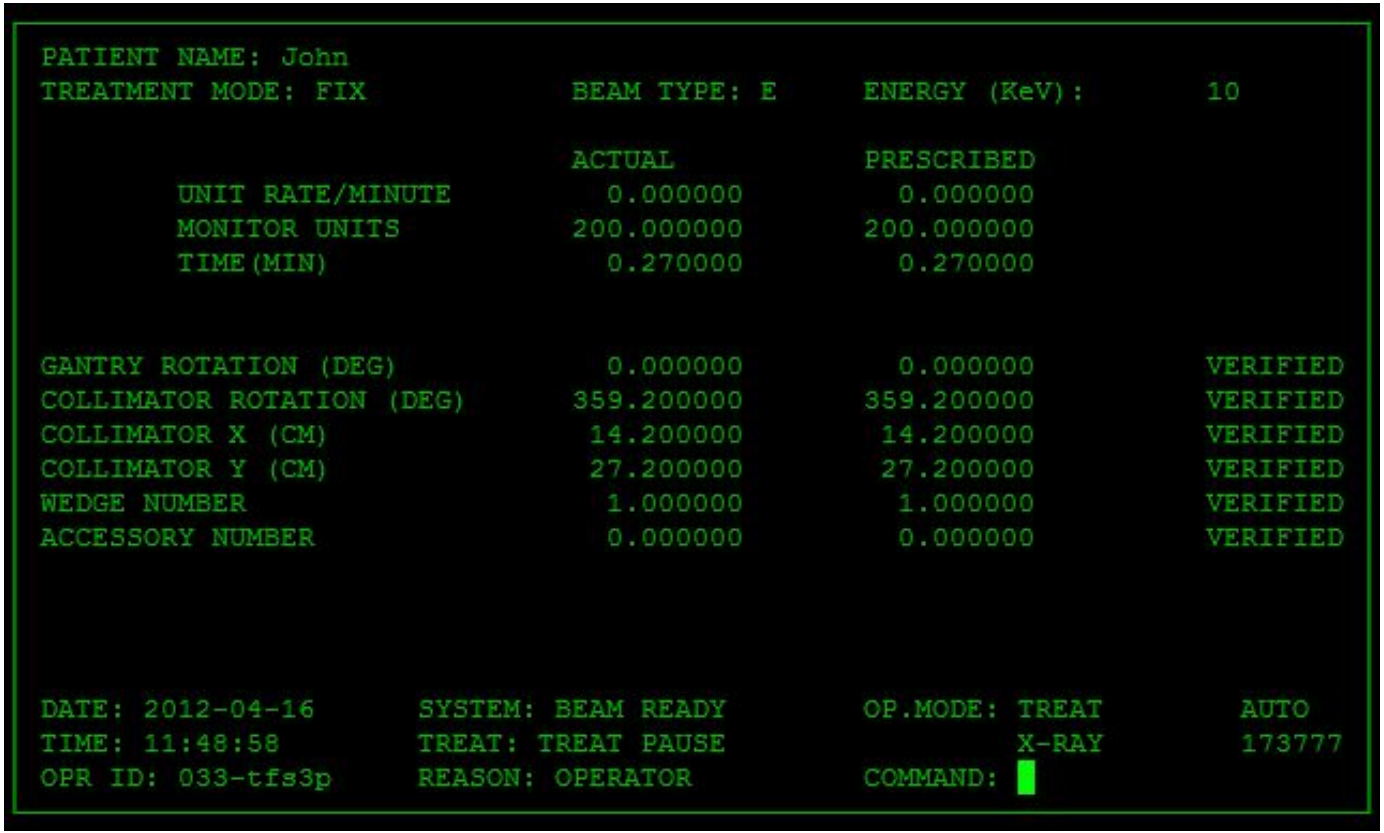
Therac-25

one of the most (in)famous software bugs in history

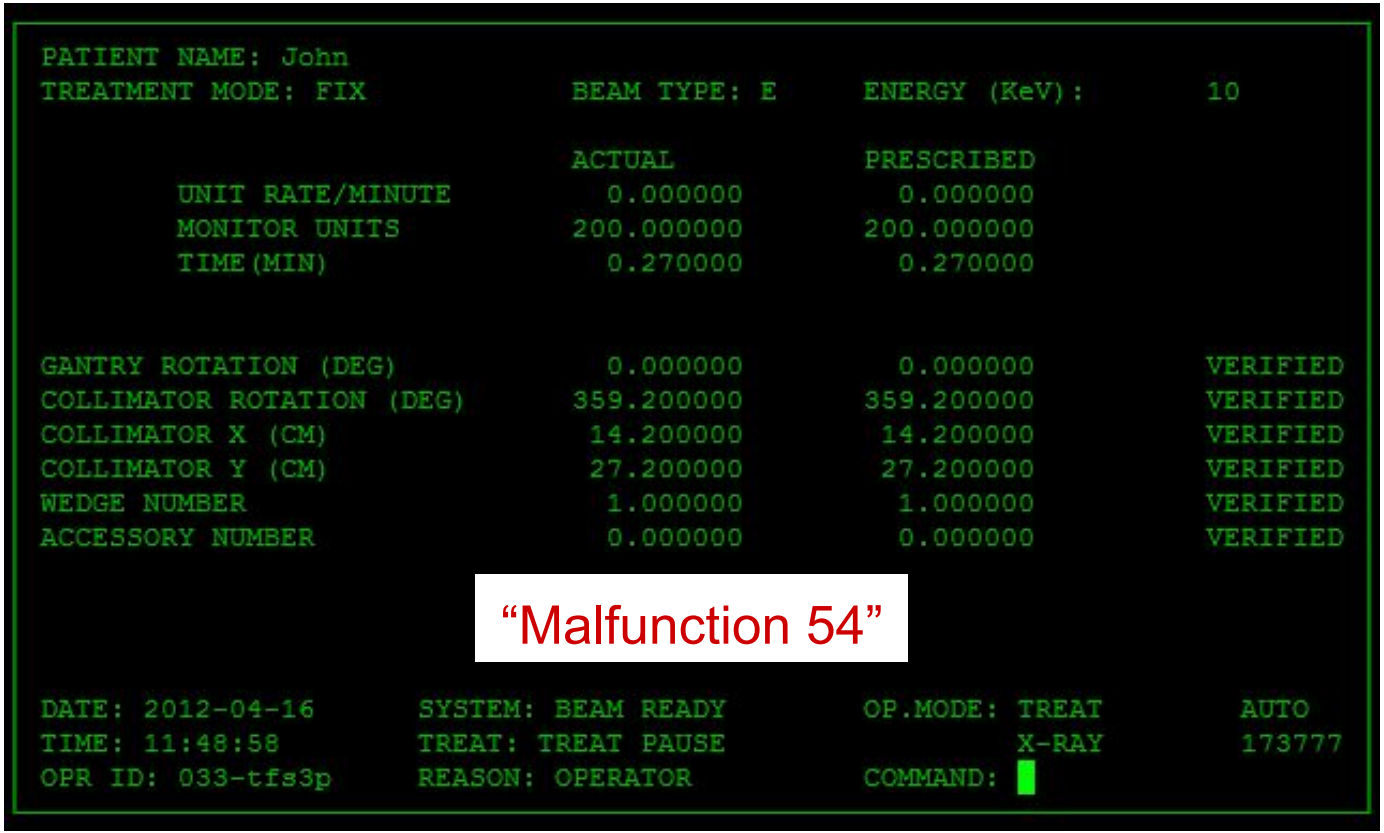
Radiation therapy machine (1985-1987)

- Under seemingly random conditions it would give 100+x the intended radiation dose to patients
- manufacturers repeatedly denied any fault and the machine’s use continued even after the first overdoses
- At least 6 serious incidents, 3 deaths

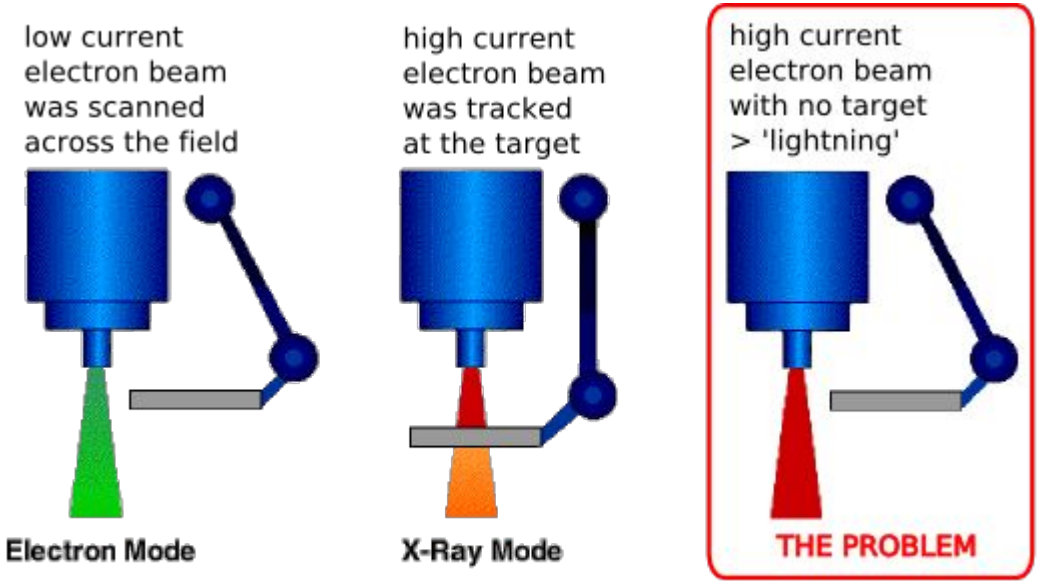
Therac-25


PATIENT NAME: John
TREATMENT MODE: FIX
BEAM TYPE: E
ENERGY (KeV): 10
ACTUAL
PRESCRIBED
UNIT RATE/MINUTE 0.000000 0.000000
MONITOR UNITS 200.000000 200.000000
TIME(MIN) 0.270000 0.270000
GANTRY ROTATION (DEG) 0.000000 0.000000 VERIFIED
COLLIMATOR ROTATION (DEG) 359.200000 359.200000 VERIFIED
COLLIMATOR X (CM) 14.200000 14.200000 VERIFIED
COLLIMATOR Y (CM) 27.200000 27.200000 VERIFIED
WEDGE NUMBER 1.000000 1.000000 VERIFIED
ACCESSORY NUMBER 0.000000 0.000000 VERIFIED
DATE: 2012-04-16
SYSTEM: BEAM READY
OP.MODE: TREAT AUTO
TIME: 11:48:58
TREAT: TREAT PAUSE
X-RAY 173777
OPR ID: 033-tfs3p
REASON: OPERATOR
COMMAND:

Therac-25


PATIENT NAME: John
TREATMENT MODE: FIX
BEAM TYPE: E
ENERGY (KeV): 10
ACTUAL
PRESCRIBED
UNIT RATE/MINUTE 0.000000 0.000000
MONITOR UNITS 200.000000 200.000000
TIME(MIN) 0.270000 0.270000
GANTRY ROTATION (DEG) 0.000000 0.000000 VERIFIED
COLLIMATOR ROTATION (DEG) 359.200000 359.200000 VERIFIED
COLLIMATOR X (CM) 14.200000 14.200000 VERIFIED
COLLIMATOR Y (CM) 27.200000 27.200000 VERIFIED
WEDGE NUMBER 1.000000 1.000000 VERIFIED
ACCESSORY NUMBER 0.000000 0.000000 VERIFIED
"Malfunction 54"
DATE: 2012-04-16
SYSTEM: BEAM READY
OP.MODE: TREAT
AUTO
TIME: 11:48:58
TREAT: TREAT PAUSE
X-RAY
173777
OPR ID: 033-tfs3p
REASON: OPERATOR
COMMAND:

Therac-25


low current
electron beam
was scanned
across the field
Electron Mode
high current
electron beam
was tracked
at the target
X-Ray Mode
high current
electron beam
with no target
> 'lightning'
THE PROBLEM

Therac-25

The bug was detectable in software!

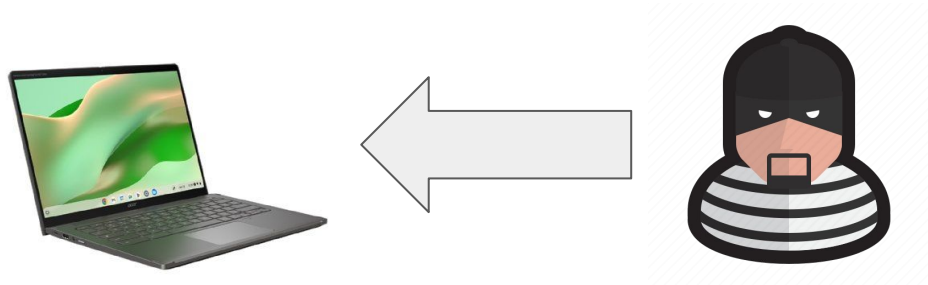
Malfunctions/errors were common when operating the terminal; operators learned to ignore them

Would verification help?

Yes: by making a known bad state unreachable

2. Security

If the software is vulnerable to attack, you may not have considered all the ways it could be exploited



Access control bugs

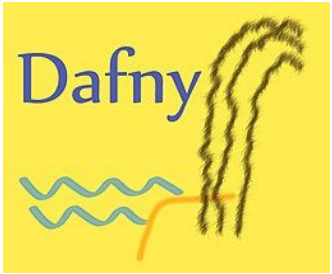
Expose critical customer or user data to malicious actors!

Access control bugs

Cloud providers

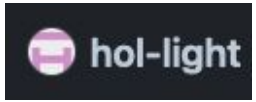
- One serious bug would be enough to destroy trust in a provider

AWS is investing millions in verification tools (including using Z3 and Dafny) for AWS S3 and IAM, AWS Encryption SDK, and other projects)



Low-level cryptographic libraries

- if these are incorrect, it can take down the whole security foundation of the internet!
- Signal messaging app: verification effort for core messaging protocol going back to 2017
- AWS-LibCrypto: open source SSL/OpenSSL implementation that is proved using Coq, HOLLight, and other tools.



XZ Utils

A recent security breach (2024):

- User by the name of “Jia Tan” builds reputation, acquires admin rights and hijacks repo
- Found by a Microsoft employee by mistake

- Was assigned the highest possible vulnerability score (CVSS 10.0)

Video: https://www.youtube.com/watch?v=aoag03mSuXQ

Other misc examples

Galois, inc. has several projects in this area including the

SAW verification tools and the

Cryptol domain-specific language

3. Cost

A bug is very expensive for your company/organization

- The financial investment – companies are willing to invest millions of dollars into tools which might prevent a future critical bug from happening

Other examples: blockchain technology

https://immunefi.com/immunefi-top-10/

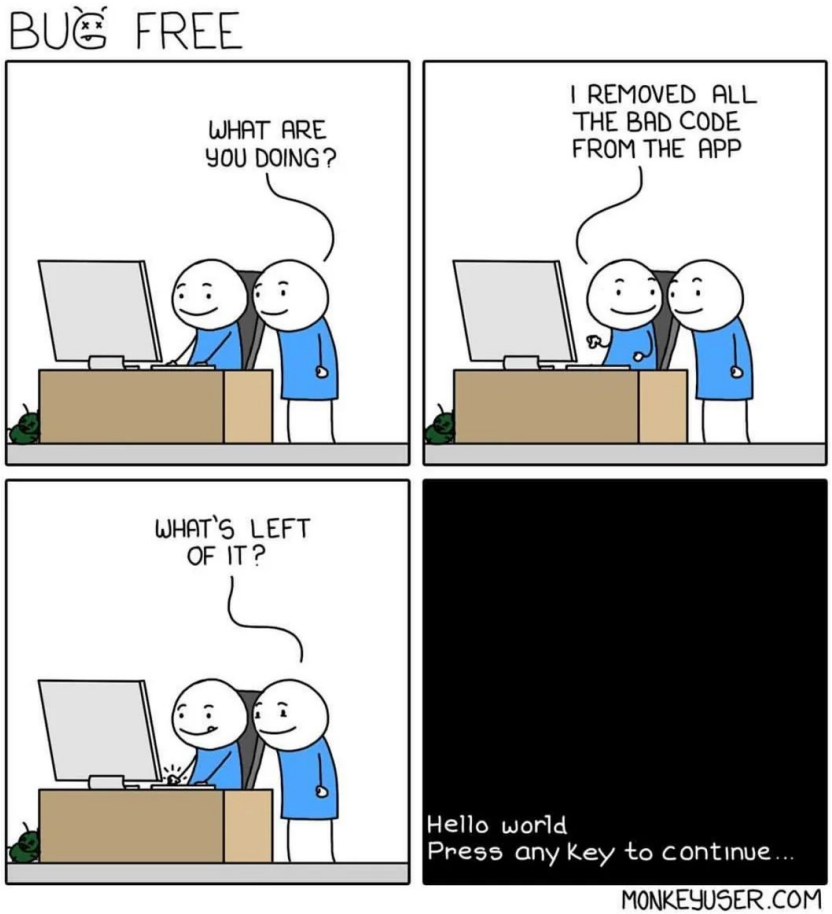
Top vulnerabilities in smart contracts

“The Beanstalk Logic Error Bugfix Review showcases an example of a missing input validation vulnerability. The Beanstalk Token Facet contract had a vulnerability in the transferTokenFrom() function, where the msg.sender’s allowance was not properly validated during an EXTERNAL mode transfer. This flaw allowed an attacker to transfer funds from a victim’s account who had previously granted approval to the Beanstalk contract.”

Lots of startups, e.g. Cubist, Veridise

Still not convinced?

- Hope for a brighter future?

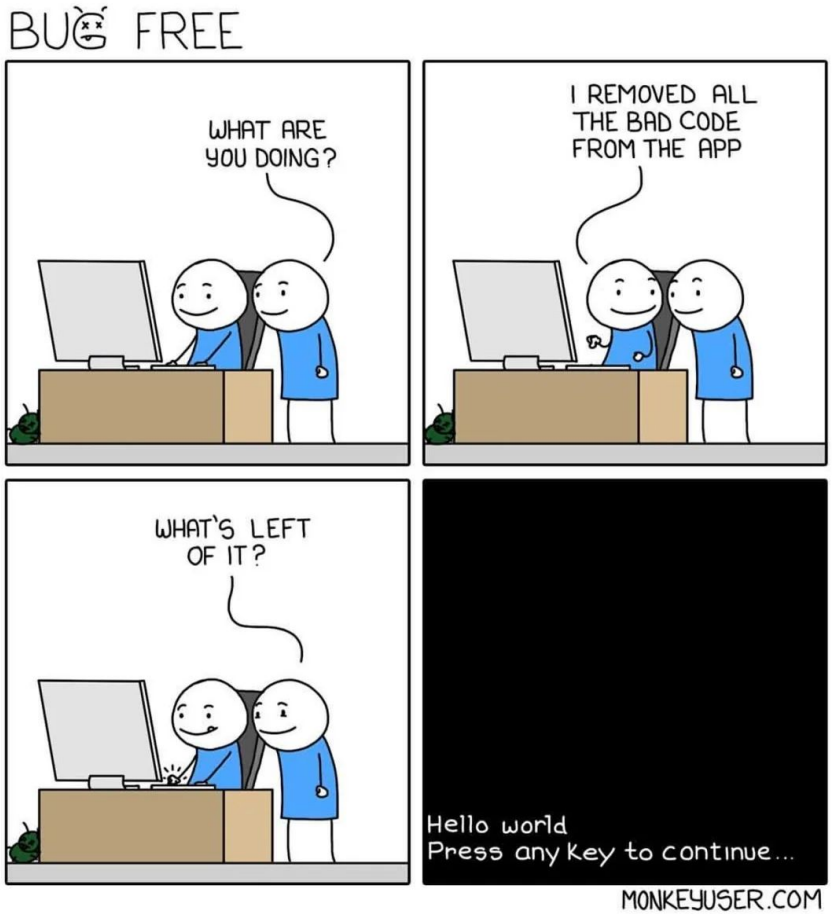


Still not convinced?

- Hope for a brighter future?

[FM x AI 2025]

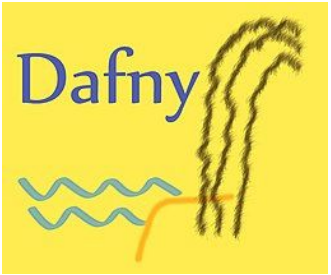
[Galois], Google DeepMind, etc.



Interactive verification tools

In this course, we will be using Dafny, a verification-aware programming language from Microsoft Research*

* now developed, funded, and widely used internally at Amazon



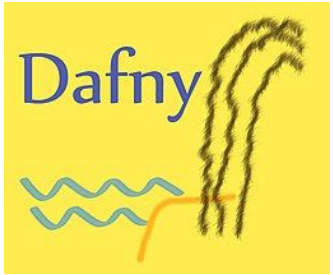
Why Dafny?

- It's modern (actively developed)

- It's used in real industry applications

- It can *cross-compile* to other languages: such as C#, Go, Python, Java, and JavaScript.

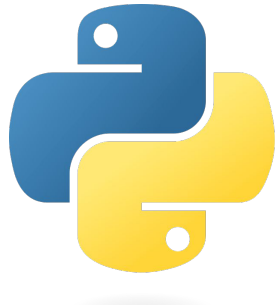
- It has a good IDE (VSCode extension)

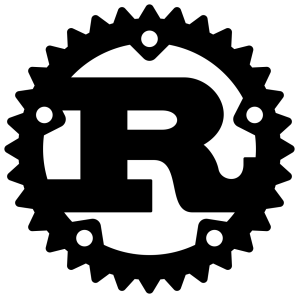


Verification tools in other popular languages?

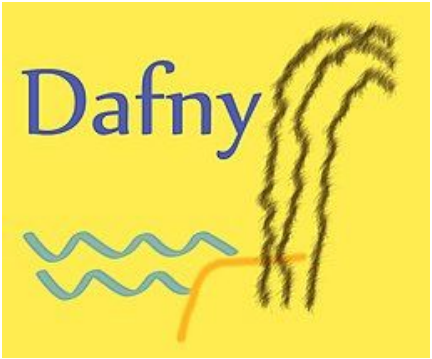
Yes!

SEE: Detailed list in lecture notes





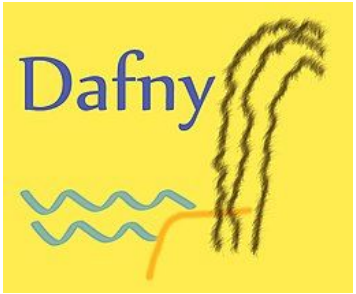
Before we get started…



Note: quick advertisement for ECS 261

I also teach a class ECS 261: that goes into Dafny programming in a lot more detail!

Verification and proofs at compile time



There is some overlap with 189C. We also have a final project which applies Dafny to write a piece of real verified software for your application domain of choice.